Лабораторна робота №1. Налаштування локального середовища розробки: Node.js, Vite, ESLint, Prettier

> **Примітка щодо актуальності.** Документ підготовлено станом на 3 листопада 2026 року: Node.js 26 LTS, Vite 8, React 19, ESLint 10, Prettier 3. Версії, запитання `create-vite` і назви меню можуть змінитися; у разі відмінностей орієнтуйтеся на офіційну документацію (§8).

1. Мета роботи

Підготувати локальне середовище React-розробки для наступних лабораторних: Node.js, проєкт на Vite, ESLint і Prettier, налаштований редактор, React Developer Tools, AI-агент, який знає правила проєкту з `AGENTS.md`, і деплой на Vercel.

Очікуваний результат: React-застосунок на Vite запускається командою `npm run dev`; `npm run lint`, `npm run format:check` і `npm run build` проходять без помилок; редактор форматує файл під час збереження й підкреслює помилки ESLint; AI-агент читає `AGENTS.md` і запускає перевірки, перш ніж звітувати; код — у репозиторії на GitHub, а застосунок відкривається на Vercel.

2. Цілі роботи

- встановити Node.js LTS і пояснити, навіщо фронтенд-проєкту Node.js, npm і `package.json`;
- створити React-проєкт шаблоном Vite й розібратися в його структурі та скриптах;
- побачити, як працюють dev-сервер з HMR (React Fast Refresh) і production-збірка;
- налаштувати ESLint із правилами хуків React і Prettier так, щоб вони не конфліктували;
- налаштувати редактор: форматування під час збереження, підказки ESLint, спільні налаштування в репозиторії;
- встановити React Developer Tools і знайти в них компонент та його стан;
- написати `AGENTS.md` для React-проєкту й перевірити, що агент його дотримується;
- зберегти проєкт у Git і на GitHub, розгорнути його на Vercel.

3. Завдання лабораторної роботи

1. Встановити Node.js LTS, Git, редактор і React Developer Tools (§4.1).
2. Створити проєкт шаблоном Vite «React + JavaScript» з ESLint і зробити перший коміт (§4.2).
3. Запустити dev-сервер і production-збірку, розібрати структуру проєкту (§4.3).
4. Додати Prettier і `eslint-config-prettier`, скрипти `format` і `format:check` (§4.4).
5. Налаштувати редактор (§4.5).
6. Перевірити інструменти на навмисних помилках (§4.6).
7. Написати `AGENTS.md` і перевірити агента двома запитами (§4.7).
8. Опублікувати репозиторій на GitHub, розгорнути застосунок на Vercel (§4.8) і надіслати звіт (§5).

4. Хід виконання роботи

4.1. Підготовка середовища

1. **Node.js.** Потрібен Node.js 22.13 або новіший. ESLint 10 працює на Node.js 20.19+, 22.13+ або 24+, але підтримку Node.js 20 завершено у квітні 2026 року. Рекомендовано актуальну LTS-версію — Node.js 26 (LTS з 28 жовтня 2026 року); Node.js 24 з курсу «HTML, CSS, JS/TS» теж підходить. Перевірте версії:

   ```
   node --version
   npm --version
   git --version
   ```

   Якщо проєкти потребують різних версій Node.js, зручно користуватися менеджером версій: fnm, nvm або Volta.
2. **Редактор.** Підійде редактор з курсу «HTML, CSS, JS/TS»: VS Code, Cursor, Windsurf або WebStorm. У VS Code, Cursor і Windsurf встановіть розширення **ESLint** (`dbaeumer.vscode-eslint`) і **Prettier - Code formatter** (`esbenp.prettier-vscode`). WebStorm підтримує обидва інструменти без розширень.
3. **Браузер.** Chrome або Edge з розширенням **React Developer Tools** (є також для Firefox). Після встановлення в DevTools з'являються вкладки ⚛ **Components** і ⚛ **Profiler** — на сторінках, зроблених на React.
4. **AI-агент.** Той самий, що в курсі «HTML, CSS, JS/TS», з режимом, у якому агент сам змінює файли (зазвичай Agent). Чи читає він `AGENTS.md`, — у таблиці §4.7.
5. **Vercel.** Акаунт з курсу «HTML, CSS, JS/TS», прив'язаний до GitHub. Якщо акаунта немає, створіть його на vercel.com через вхід з GitHub.

4.2. Створення проєкту

1. У теці, де лежатимуть ваші проєкти, виконайте:

   ```
   npm create vite@latest
   ```

   Якщо npm запитає дозвіл установити `create-vite`, відповідайте `y`. Далі відповідайте так:

Запитання	Відповідь
Project name	`react-lab-1`
Select a framework	React
Select a variant	JavaScript
Which linter to use?	ESLint
Install with npm and start now?	No

   Чому так. TypeScript з'явиться в ЛР 19, React Compiler — у ЛР 15, тож зараз — JavaScript без компілятора. За замовчуванням шаблон пропонує Oxlint — швидкий лінтер на Rust, який реалізує лише частину правил і плагінів ESLint. Курс використовує ESLint: правила хуків (`eslint-plugin-react-hooks`) розробляє сама команда React.

2. Встановіть залежності:

   ```
   cd react-lab-1
   npm install
   ```

   npm може повідомити про вразливості рівня moderate в залежностях ESLint. Вони стосуються лише інструментів розробки й не потрапляють у збірку. **Не запускайте `npm audit fix --force`:** він «виправляє» вразливості, понижуючи версії пакетів. У цьому проєкті він замінив би `eslint-config-prettier` 10 на 6.15, а `@eslint/js` 10 — на 9, і жодної вразливості не виправив би.

3. Створіть у корені проєкту файл `.gitattributes` з одним рядком:

   ```
   * text=auto eol=lf
   ```

   Git на Windows зазвичай перетворює кінці рядків на CRLF, коли записує файли в робочу теку. Prettier очікує LF, тож у свіжому клоні репозиторію (у викладача чи на іншому комп'ютері) `npm run format:check` позначив би кожен файл. Цей рядок змушує Git зберігати й видавати файли з LF.

4. Зробіть перший коміт. Файл `.gitignore` шаблон уже містить:

   ```
   git init -b main
   git add .
   git commit -m "chore: scaffold React app with Vite"
   ```

4.3. Структура проєкту й скрипти

Файл чи тека	Що це
`index.html`	сторінка, з якої починає Vite: `<div id="root">` і `<script type="module" src="/src/main.jsx">`
`src/main.jsx`	точка входу: `createRoot(...).render(...)` малює `<App />` всередині `<StrictMode>`
`src/App.jsx`	кореневий компонент; у шаблоні — лічильник на `useState`
`src/App.css`, `src/index.css`	стилі
`src/assets/`	файли, які імпортують з коду; у збірку потрапляють з хешем у назві
`public/`	файли, які копіюються в збірку як є
`vite.config.js`	налаштування Vite з плагіном `@vitejs/plugin-react`
`eslint.config.js`	налаштування ESLint у форматі flat config
`package.json`, `package-lock.json`	залежності й скрипти; lock-файл фіксує точні версії всіх пакетів

Команда	Що робить
`npm run dev`	dev-сервер на http://localhost:5173 з HMR
`npm run build`	production-збірка в теку `dist/`
`npm run preview`	локальний перегляд зібраного `dist/` на http://localhost:4173
`npm run lint`	ESLint для всього проєкту

1. Запустіть `npm run dev` і відкрийте http://localhost:5173. Відкрийте DevTools → ⚛ **Components**: виберіть `App` і знайдіть його стан у розділі **hooks**. Натисніть кнопку лічильника кілька разів і подивіться, як змінюється стан. Кнопкою вибору елемента у вкладці Components знайдіть компонент за елементом сторінки.
2. Зупиніть сервер (`Ctrl+C`). Виконайте `npm run build`: Vite виведе список файлів `dist/` з розмірами. Запустіть `npm run preview` і відкрийте http://localhost:4173.
3. Теки `node_modules/` і `dist/` є в `.gitignore`: їх не комітять, бо npm і Vite відтворюють їх з `package-lock.json` і з коду.

4.4. Prettier

1. Встановіть Prettier і `eslint-config-prettier`. Prettier радить фіксувати точну версію (`--save-exact`): навіть patch-реліз може трохи змінити форматування, і в учасників команди воно розійдеться.

```
npm install --save-dev --save-exact prettier
npm install --save-dev eslint-config-prettier
```

2. Створіть у корені `.prettierrc.json`:

```
{
  "semi": false,
  "singleQuote": true
}
```

   Це стиль шаблону Vite: без крапок з комою, з одинарними лапками. Тож Prettier не змінить у файлах шаблону жодного рядка. Решта параметрів — типові: ширина рядка 80, відступ 2 пробіли. Файли з `.gitignore` (`node_modules/`, `dist/`) Prettier 3 пропускає сам, тож окремий `.prettierignore` не потрібен.
3. Додайте в `package.json` до `"scripts"` два рядки:

```
"format": "prettier . --write",
"format:check": "prettier . --check"
```

4. Підключіть `eslint-config-prettier` в `eslint.config.js` — імпортом і останнім елементом масиву:

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import eslintConfigPrettier from 'eslint-config-prettier/flat'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true } },
    },
  },
  eslintConfigPrettier,
])
```

   `eslint-config-prettier` вимикає правила ESLint, які сперечаються з Prettier про форматування. Він стоїть останнім, бо пізніші елементи масиву перекривають ранніші. У поточних наборах правил таких немає, тож зараз він нічого не змінює — і

захищає від конфліктів, коли ви додасте плагін зі стильовими правилами. Перевірити це можна командою, яку пакет має для цього:

```
npx eslint-config-prettier src/main.jsx
```

5. Запустіть `npm run lint` і `npm run format:check`: обидві команди мають пройти без помилок. Зробіть коміт `chore: add Prettier`.

4.5. Редактор

VS Code, Cursor, Windsurf. Створіть теку `.vscode/` з двома файлами. `extensions.json` пропонує розширення кожному, хто відкриє проєкт:

```
{
  "recommendations": ["dbaeumer.vscode-eslint", "esbenp.prettier-vscode"]
}
```

`settings.json` вмикає Prettier під час збереження файлу й виправлення ESLint, які можна застосувати автоматично:

```
{
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  }
}
```

`.gitignore` шаблону ігнорує всю `.vscode/`, крім `extensions.json`. Додайте під рядком `!.vscode/extensions.json` ще один:

```
!.vscode/settings.json
```

Перевірте `git status`: обидва файли `.vscode/` мають бути серед нових.

WebStorm. У **Settings** → **Languages & Frameworks** → **JavaScript** → **Code Quality Tools** → **ESLint** виберіть **Automatic ESLint configuration** і позначте **Run eslint --fix on save**. У **Settings** → **Languages & Frameworks** → **JavaScript** → **Prettier** виберіть **Automatic Prettier configuration** і позначте **Run on save**. Налаштування WebStorm живуть у `.idea/`, яку шаблон ігнорує, тож у звіт додайте знімок екрана цих двох сторінок.

Зробіть коміт `chore: share editor settings`.

4.6. Перевірка інструментів

Ці перевірки нічого не залишають у проєкті: після кожної поверніть файли до останнього коміту (`git status` має бути чистим).

П1. Fast Refresh. Запустіть `npm run dev`, натисніть лічильник до 5. У `src/App.jsx` замініть текст `Get started` своїм ім'ям і збережіть файл. Сторінка оновиться без перезавантаження, а лічильник залишиться на 5: React Fast Refresh замінив код компонента й зберіг його стан. Поверніть текст.

П2. ESLint. Створіть тимчасовий файл `src/LintDemo.jsx`:

```
import { useEffect, useState } from 'react'

export default function LintDemo({ items }) {
  const [total, setTotal] = useState(0)
  const label = 'Разом'

  if (items.length === 0) {
    const [message] = useState('Порожньо')
    return <p>{message}</p>
  }

  useEffect(() => {
    setTotal(items.length)
  }, [items])

  return <p>{total}</p>
}

export function formatTotal(total) {
  return `Разом: ${total}`
}
```

Редактор підкреслить помилки ще до запуску лінтера. Запустіть `npm run lint`: має бути 5 помилок від 4 правил. Зробіть знімок екрана для звіту й запишіть, що означає кожне правило. Видаліть `src/LintDemo.jsx`.

П3. Prettier. У `src/App.jsx` замініть `import './App.css'` на `import "./App.css";` і збережіть **без форматування**: у VS Code — **File: Save without Formatting** (`Ctrl+K S`), у WebStorm — тимчасово зніміть **Run on save**. Запустіть `npm run format:check`: Prettier назве `src/App.jsx`. Тепер збережіть файл звичайно (`Ctrl+S`): лапки й крапка з комою зміняться, а `npm run format:check` пройде.

П4. Dev проти збірки. Відкрийте DevTools → **Network**, фільтр **JS**, і перезавантажте сторінку спершу на `npm run dev` (http://localhost:5173), потім на `npm run preview` після

`npm run build` (http://localhost:4173). Порівняйте, скільки JS-файлів завантажує сторінка в кожному режимі і як вони називаються.

4.7. AGENTS.md для React-проєкту

`AGENTS.md` — звичайний Markdown-файл у корені репозиторію з інструкціями для AI-агента: команди перевірки, структура проєкту, правила коду. Агент читає його на початку кожної сесії, тож правила не треба повторювати в кожному запиті.

Для React такий файл особливо корисний. Агенти навчалися на великій кількості старого React-коду й часто пропонують те, чого в сучасному React уже немає або що не радять: класові компоненти, `ReactDOM.render`, Create React App, `propTypes`, зайві `useEffect`. `AGENTS.md` прямо називає версії й правила вашого проєкту.

Які агенти читають `AGENTS.md`:

Агент	Як підхоплює `AGENTS.md`
GitHub Copilot у VS Code	автоматично з кореня робочої теки; керує налаштування `chat.useAgentsMdFile`, увімкнене за замовчуванням
Cursor, Windsurf, Codex, Junie (JetBrains)	автоматично
Claude Code	автоматично, якщо в проєкті немає `CLAUDE.md`; якщо є — Claude Code читає лише `CLAUDE.md`, тож додайте в нього рядок `@AGENTS.md`
Gemini CLI	після налаштування: у `.gemini/settings.json` — `{ "context": { "fileName": ["AGENTS.md", "GEMINI.md"] } }`

Як писати `AGENTS.md`:

- Пишіть те, чого агент не дізнається з коду сам: команди перевірки, версії, заборони, рішення, ухвалені в проєкті.
- Одне правило — одне конкретне речення: «Run `npm run lint`», а не «пиши якісний код».
- Те, що може перевірити інструмент, хай перевіряє інструмент. `AGENTS.md` не переписує правила ESLint, а вимагає запустити `npm run lint` і не вимикати правил.
- Файл росте разом із проєктом: коли агент удруге робить ту саму помилку, допишіть правило.
- Жодних секретів: файл лежить у публічному репозиторії.

1. Створіть у корені проєкту `AGENTS.md`. Почніть із шаблону й допишіть власні правила, якщо вони у вас є. Шаблон англійською: так його однаково розуміють усі агенти, а

українська залишається для текстів інтерфейсу.

```
# AGENTS.md

A single-page app on React 19 and Vite 8, in JavaScript with JSX. No
TypeScript yet. User-facing text is in Ukrainian.

## Commands

- `npm install`: install dependencies
- `npm run dev`: dev server at http://localhost:5173
- `npm run lint`: ESLint
- `npm run format`: format all files with Prettier
- `npm run format:check`: check formatting without changing files
- `npm run build`: production build into `dist/`

A task is done only when `npm run lint`, `npm run format:check` and
`npm run build` pass. Run all three and report their results.

## Project structure

- `index.html`: the page Vite serves; the app mounts into `<div id="root">`
- `src/main.jsx`: entry point; renders `<App />` inside `<StrictMode>`
- `src/App.jsx`: root component
- `src/components/`: one component per file, named in PascalCase
  (`ProfileCard.jsx`)
- `public/`: static files, copied to the build as is

## React rules

- Function components and hooks only. No class components.
- Use current React 19 APIs: `createRoot`, not `ReactDOM.render`. No
  `propTypes`, and no `defaultProps` on function components (use default
  parameter values instead).
- Follow the Rules of Hooks: call hooks only at the top level of a
component
  or a custom hook, never in a condition, a loop or after an early return.
- Keep rendering pure. Compute values from props and state during render
  instead of copying them into state with `useEffect`.
- Give list items a stable `key` from the data, never the array index.
- Use semantic HTML: `<button>` for actions, `<a href>` for navigation, a
  `<label>` for every form field, `alt` on every `<img>`.
- Don't use `dangerouslySetInnerHTML`.

## Dependencies and configuration

- Don't add, remove or upgrade npm packages without asking first, and say
why
```

```
  a package is needed.
- Don't run `npm audit fix --force`.
- Don't change `eslint.config.js`, `.prettierrc.json`, `vite.config.js` or
  the scripts in `package.json` unless the task is about them.
- Never turn off ESLint rules or add `eslint-disable` comments. Fix the
cause.

## How to work

- Before editing, read the relevant files and list the files you will
change.
- Keep each change small and limited to the task.
- After editing, summarise what changed and show the check results.
- Never put secrets (API keys, tokens, passwords) in the code. Every
  `VITE_*` variable ends up in the public bundle.
```

2. Запустіть `npm run format:check`: Prettier перевіряє й Markdown. Якщо він назве `AGENTS.md`, виконайте `npm run format`. Зробіть коміт `docs: add AGENTS.md`.
3. **Запит 1 — чи прочитав агент файл.** Почніть нову сесію агента в режимі без редагування (Ask чи Plan) і надішліть:

   > Не змінюй жодного файлу. Коротко перекажи правила цього проєкту: якими командами ти перевірятимеш свою роботу, яких правил React маєш дотримуватися і що тобі заборонено робити без мого дозволу.

   Агент має назвати команди й правила саме з вашого `AGENTS.md`. Якщо відповідь загальна, без ваших команд, — агент файлу не бачить: перевірте таблицю вище й налаштування агента.
4. **Запит 2 — зміна за правилами.** Перемкніть агента в режим Agent і надішліть, підставивши свої дані:

   > Заміни демонстраційний вміст `src/App.jsx` на картку «Про мене». Ім'я: <ваше ім'я>. Група: <ваша група>. Технології, які я хочу опанувати в цьому курсі: <три технології>. На картці — ім'я, група, список технологій і кнопка «Подобається» з лічильником натискань. Картка — окремий компонент `ProfileCard`, дані він отримує через props. Видали стилі й зображення шаблону, які стануть непотрібними. Спочатку покажи план і список файлів, які зміниш.

5. **Перевірте роботу агента** — за `git diff`, у терміналі й у браузері:

Що перевірити	Правило `AGENTS.md`
план до змін	агент перелічив файли, перш ніж редагувати
розташування	компонент лежить у `src/components/ProfileCard.jsx`

Що перевірити	Правило `AGENTS.md`
React	функційний компонент, стан лічильника — `useState`, `key` у списку не з індексу
семантика	лічильник — `<button>`, а не `<div onClick>`
залежності й конфіг	`package.json`, `eslint.config.js`, `.prettierrc.json`, `vite.config.js` не змінено
перевірки	агент запустив `npm run lint`, `npm run format:check` і `npm run build` і показав результат

Відкрийте сторінку й у ⚛ **Components** знайдіть `ProfileCard`: його props і стан лічильника. Закомітьте зміну агента без виправлень: `feat: add profile card`. Якщо агент порушив правило, виправте це власноруч окремим комітом, наприклад `fix: use button for the like counter`, і подумайте, яке речення в `AGENTS.md` запобігло б цьому наступного разу.

4.8. GitHub і Vercel

1. Створіть на GitHub порожній репозиторій `react-lab-1` — без README, `.gitignore` і ліцензії: вони вже є в проєкті або не потрібні.
2. Підключіть його й відправте код:

```
git remote add origin https://github.com/<username>/react-lab-1.git
git push -u origin main
```

3. Перевірте репозиторій у браузері: є `AGENTS.md`, `.gitattributes`, `.prettierrc.json` і `.vscode/`, немає `node_modules/` і `dist/`. Історія комітів:
   1. `chore: scaffold React app with Vite` — шаблон без змін і `.gitattributes`;
   2. `chore: add Prettier`;
   3. `chore: share editor settings`;
   4. `docs: add AGENTS.md`;
   5. `feat: add profile card` — зміна агента без виправлень;
   6. виправлення, якщо агент порушив правила `AGENTS.md`.
4. Увійдіть на vercel.com → **Add New** → **Project** і імпортуйте репозиторій `react-lab-1`. Vercel сам розпізнає Vite: перед **Deploy** перевірте, що **Framework Preset** — **Vite**, а **Output Directory** — `dist`. Vercel встановить залежності з `package-lock.json`, виконає `npm run build` і опублікує теку `dist/`. Тому `dist/` і не комітять.
5. Відкрийте адресу `https://<project>.vercel.app/`: має відкритися картка «Про мене». Кожен наступний `git push` у `main` створює новий деплой — перевірте це на будь-якій наступній зміні.

5. Вимоги до звіту

Тема листа:

```
[React] ЛР-1: Налаштування середовища розробки
```

У листі вкажіть репозиторій і адресу застосунку на Vercel:

```
GitHub repository: https://github.com/<username>/react-lab-1
Vercel: https://<project>.vercel.app/
```

Звіт повинен містити:

1. результат `node --version`, `npm --version` і `git --version`; редактор і встановлені розширення (для WebStorm — знімки екрана налаштувань з §4.5);
2. знімок екрана застосунку з відкритою вкладкою ⚛ **Components**;
3. знімок екрана `npm run lint` з помилками П2 і коротко — що означає кожне з 4 правил;
4. знімок екрана `npm run lint`, `npm run format:check` і `npm run build` без помилок після всіх комітів;
5. назву й версію AI-агента, модель, дату роботи; звідки агент бере правила проєкту;
6. короткий висновок: що налаштовано й перевірено, що було складним.

6. Усний захист

Формат здачі передбачає усну співбесіду за результатами роботи. Питання нижче — типові питання співбесіди frontend-розробника з тем цієї роботи. Відповідайте на прикладах власного проєкту. Викладач може попросити показати відповідь у коді, терміналі чи DevTools.

Питання для підготовки:

Node.js і npm

- Навіщо фронтенд-проєкту Node.js, якщо застосунок працює в браузері?
- Що таке LTS-версія Node.js? Звідки відомо, яку версію Node.js вимагає Vite чи ESLint?
- Що зберігається в `package.json`, а що — в `package-lock.json`? Чому lock-файл комітять, а `node_modules/` — ні?
- Чим `dependencies` відрізняються від `devDependencies`? Чому `react` — в одному, а `eslint` — в іншому?
- Що означають версії `^19.3.1` і `3.9.10` без `^`? Навіщо Prettier встановлюють з `--save-exact`?

- Що показує `npm audit`? Чому `npm audit fix --force` небезпечний?

Vite

- Що робить Vite у режимі dev, а що — під час `vite build`?
- Чому dev-сервер не збирає застосунок в один файл? Що ви побачили у вкладці Network у П4?
- Що таке HMR і React Fast Refresh? Чому в П1 лічильник не скинувся?
- Навіщо хеш у назвах файлів у `dist/assets/`?
- Чим `public/` відрізняється від `src/assets/`?
- Чому `index.html` лежить у корені проєкту, а не в `public/`?
- Що робить `@vitejs/plugin-react`?

React-проєкт

- Що роблять `createRoot` і `render` у `src/main.jsx`?
- Навіщо `<StrictMode>`? Що він змінює в режимі dev і чи діє в production?
- Що таке JSX і хто перетворює його на JavaScript?
- Що показує вкладка Components у React DevTools? Як знайти компонент за елементом на сторінці?

ESLint і Prettier

- Чим лінтер відрізняється від форматера? Наведіть помилку, яку знайде ESLint, але не Prettier.
- Що таке flat config? З яких частин складається ваш `eslint.config.js`?
- Що перевіряють правила `eslint-plugin-react-hooks` з П2? Чому хук не можна викликати в умові чи після раннього `return`?
- Чому `react-refresh/only-export-components` не дозволяє експортувати функцію поруч із компонентом?
- Навіщо `eslint-config-prettier` і чому він стоїть останнім?
- Чим Oxlint відрізняється від ESLint? Чому шаблон Vite пропонує його за замовчуванням?
- Навіщо комітити `.prettierrc.json` і `.vscode/settings.json`, якщо кожен може налаштувати свій редактор?
- Що робить `.gitattributes`? Що показав би `npm run format:check` у свіжому клоні на Windows без нього?

Деплой на Vercel

- Що запускає новий деплой на Vercel? Що Vercel робить з репозиторієм: яку команду виконує і яку теку публікує?

- Чому для деплою не потрібно комітити `dist/`?
- Чим застосунок на Vercel відрізняється від `npm run preview`, а чим — від `npm run dev`?

AI-агент і AGENTS.md

- Що таке `AGENTS.md` і коли агент його читає? Як ви перевірили, що ваш агент його прочитав?
- Чим правило в `AGENTS.md` відрізняється від того самого речення в запиті?
- Чому правило «запусти `npm run lint`» надійніше за правило «не порушуй правил хуків»?
- Які застарілі підходи React агенти пропонують найчастіше і чому?
- Яких правил `AGENTS.md` агент не дотримався в запиті 2? Як ви це помітили?
- Чому значення змінних `VITE_*` не можна вважати секретами?

7. Критерії готовності

Роботу зараховано після захисту (§6); вона оцінюється в 1 бал.

Критерій	Мінімальна вимога
Node.js	Node.js 22.13 або новіший; `node`, `npm` і `git` працюють
Проєкт	шаблон Vite «React + JavaScript» з ESLint; `npm run dev`, `npm run build` і `npm run preview` працюють
ESLint	`npm run lint` без помилок; редактор підкреслює помилки
Prettier	`.prettierrc.json`, скрипти `format` і `format:check`, `eslint-config-prettier`; `npm run format:check` без помилок
Редактор	форматування під час збереження; `.vscode/` у репозиторії або знімки налаштувань WebStorm
React DevTools	встановлено; видно дерево компонентів, props і стан
AGENTS.md	у корені репозиторію; агент його читає; зміну за запитом 2 перевірено й закомічено
GitHub	коміти §4.8; немає `node_modules/` і `dist/`; є `.gitattributes`
Vercel	`https://<project>.vercel.app/` відкриває картку «Про мене» з останнього коміту `main`
Звіт	усі пункти §5

8. Список джерел

1. React. Build a React app from Scratch
2. React. Editor Setup

3. React. React Developer Tools
4. React. `<StrictMode>`
5. React. Rules of Hooks
6. React. eslint-plugin-react-hooks
7. React. You Might Not Need an Effect
8. Vite. Getting Started
9. Vite. Features
10. Vite. Static Asset Handling
11. Vite. Building for Production
12. Vite. Env Variables and Modes
13. Node.js. Download Node.js
14. Node.js. Node.js Releases
15. npm. package.json
16. npm. package-lock.json
17. npm. npm audit
18. ESLint. Configuration Files
19. Oxc. Oxlint
20. Prettier. Install
21. Prettier. Options
22. Prettier. Integrating with Linters
23. Prettier. Ignoring Code
24. Prettier. eslint-config-prettier
25. Visual Studio Marketplace. ESLint
26. Visual Studio Marketplace. Prettier - Code formatter
27. JetBrains. ESLint in WebStorm
28. JetBrains. Prettier in WebStorm
29. Git. gitattributes
30. GitHub. Configuring Git to handle line endings
31. Vercel. Vite on Vercel
32. Vercel. Deploying Git Repositories with Vercel
33. AGENTS.md. A simple, open format for guiding coding agents
34. Visual Studio Code. Custom instructions in VS Code
35. Cursor. Rules
36. Claude Code. How Claude remembers your project
37. Gemini CLI. Provide context with GEMINI.md files